oom-watch — User Manual

A Go monitoring daemon for OOM-Protect

OOM-Protect maintainers

2026-04-30

oom-watch — User Manual

`oom-watch` is the third leg of OOM-Protect. It is a Go daemon that uses atop to sample system state, evaluates thresholds, and writes a detailed Markdown forensic report **just before** the system would have been pushed off the cliff. By the time you read the report two days later, every datum needed to reason about the incident — top processes, /proc/meminfo, PSI, cgroup memory, journal tail — is already on disk.

It complements:

- **`oom-hardening.sh`** — sets the system-wide umbrella (cgroup limits on `user.slice`, `systemd-oomd`, sysctls).
- **`oom-runner.sh`** — bounds individual workloads (per-leaf cgroup scopes/services).
- **`oom-watch`** *(this tool)* — observes the umbrella in real time and produces evidence when thresholds breach.

Why it exists

After the 2026-04-28 user-manager OOM kill on host `nezha` (see `reports/Crash_Report.md`), we wanted *evidence collected while the system was still alive*. Atop's own `.log` files are post-hoc, the `journal` only has whatever was emitted, and `dmesg` records the OOM victim but not the broader context. `oom-watch` writes a self-contained Markdown report — top processes, /proc snapshots, PSI, cgroup state, recent journal — at the moment a threshold breaches, so the picture is preserved before the cascade.

Constitution / anti-bluff testing

Per the project `Constitution.md`, every test and Challenge in this toolchain must prove the product really works. `oom-watch` enforces this:

- Unit tests use real fixtures (`oom-watch/internal/atop/testdata/sample.txt`) and temp /proc trees with specific assertions.
- Challenges (`challenges/challenge-*.sh`) run the **built `oomwatch` binary** and assert against on-disk artifacts.
- One Challenge (`challenge-tests-are-not-bluffs.sh`) is a self-test of the test suite: it mutates a production line, confirms tests go red, and restores.

No `t.Skip`, no `|| true`, no swallowed exit codes. If the suite is green, the product really works.

Requirements

- Linux, kernel ≥ 5.15 (PSI), cgroup v2.
- `atop` installed and on `$PATH`. atop must run without arguments (it self-installs `/var/log/atop/` lazily, but oom-watch does not depend on those logs).
- `systemd` ≥ 248 if you want the `oom-watch.service` unit.

- Go 1.22+ to build from source.

Build

```
make oomwatch-build       # builds oom-watch/oomwatch
make oomwatch-test        # go vet + go test (anti-bluff unit suite)
make challenges           # full E2E suite (anti-bluff)
```

Or directly:

```
cd oom-watch
go build -o oomwatch ./cmd/oomwatch
go test -count=1 ./...
```

Install

The recommended path is the one-shot installer + verifier:

```
sudo make oomwatch-deploy       # or, when already root: make oomwatch-deploy
```

For the full step-by-step description of every action the deployer takes, every assertion it makes, every flag it accepts, and every failure mode and its remediation, see **`oom-watch-deployment-guide.md`** — that document is the single canonical reference for getting `oom-watch` running on a host.

In short, `make oomwatch-deploy` wraps `oomwatch-install` with: `daemon-reload`, `enable`, `restart`, a 30 s `is-active` poll, a journal check that `"atop located"` appears (proves the daemon reached its sample loop), a 60 s wait for the first report, and a forced `-one-shot` if the host is calm enough that no threshold trips. **It validates `/etc/oom-watch/config.json` with `-dry-run` BEFORE asking systemd to start the unit**, so a bad config produces the precise validator error rather than a cryptic `systemctl status: exit-code=2`, and **auto-remediates a broken installed config** by backing it up to `/etc/oom-watch/config.json.broken.<timestamp>` and copying the shipped example into place. On any failure the script's EXIT trap dumps `systemctl status`, the last 50 journal lines, the config and unit files, and a listing of the report directory.

If you prefer the legacy two-step procedure:

```
sudo make oomwatch-install
sudo systemctl enable --now oom-watch.service
```

`oomwatch-install` (and therefore `oomwatch-deploy`) places:

- `/usr/local/sbin/oomwatch`
- `/etc/oom-watch/config.json` (from `oom-watch/config/oom-watch.example.json`, only if not already present)
- `/etc/systemd/system/oom-watch.service`

- `/var/log/oom-watch/reports/` (created by daemon at start)
- `/var/lib/oom-watch/` (state)

Reports appear under `/var/log/oom-watch/reports/`, named `YYYY-MM-DDTHH-MM-SSZ-<sever-ity>.md`.

Command-line flags

```
oomwatch [flags]
  -config PATH       JSON config; omit for defaults
  -dry-run           validate config and exit (rc=0 if OK, 2 if invalid)
  -one-shot          take one sample, write a report unconditionally, exit
  -print-config      print effective config (after defaults) and exit
  -version           print version and exit
```

Configuration

JSON. The complete example is `oom-watch/config/oom-watch.example.json`. Every threshold has a `notice / warn / critical` ladder where applicable:

Field	Default	Meaning
`interval_seconds`	10	sampling cadence
`report_dir`	`/var/log/oom-watch/reports`	where reports land
`state_dir`	`/var/lib/oom-watch`	reserved for future state
`log_level`	`info`	`debug`, `info`, `warn`, `error`
`log_format`	`text`	`text` or `json`
`atop_binary`	`atop`	override if non-standard install
`thresholds.memory_used_ratio_notice`	0.80	log only
`thresholds.memory_used_ratio_warn`	0.90	first reportable severity
`thresholds.memory_used_ratio_critical`	0.95	escalate, report regardless of cooldown
`thresholds.swap_used_ratio_warn`	0.50	
`thresholds.swap_used_ratio_critical`	0.80	
`thresholds.psi_mem_full_avg10_warn`	10.0	percent, leading indicator of OOM
`thresholds.psi_mem_full_avg10_critical`	30.0	imminent thrash
`thresholds.psi_mem_some_avg10_warn`	40.0	partial stalls
`thresholds.load_per_cpu_warn`	2.0	load1 / NumCPU
`thresholds.load_per_cpu_critical`	4.0	
`report.min_interval_seconds`	60	cooldown between same-severity reports
`report.top_n_processes`	20	size of “Top processes” tables

Validation (enforced at startup):

- Notice ≤ Warn ≤ Critical for every metric.
- `memory_used_ratio_critical` must be `< 1.0` (it would otherwise never fire).

- Unknown fields are rejected — protects against silent typos.

Severity ladder and cooldown

Severity	Trigger	Cooldown
OK	nothing breached	n/a
NOTICE	any notice-level threshold	logged only, no report
WARN	any warn-level threshold	report once per `min_interval_seconds`
CRITICAL	any critical-level threshold	always reports; bypasses cooldown

An **escalation** (e.g. WARN → CRITICAL within the cooldown window) always emits a new report. A flap back down to a lower severity logs but does not re-report; the next escalation will.

Anatomy of a report

Each `.md` report contains, in order:

1. **YAML front matter** — title, host, timestamp, severity (pandoc-friendly).
2. **Triggers** — table of metrics that breached, with observed value vs. limit.
3. **Atop sample summary** — single-line summaries of MEM, SWP, PSI, CPL.
4. **Top processes by resident memory** — with PID, cmd, RSize converted to GiB/MiB.
5. **Top processes by CPU** — from atop's PRC label.
6. **Top processes — full forensic detail** — for each of the top N memory PIDs (default 10), one subsection containing the full `/proc/<pid>/cmdline` (the actual argv, not atop's truncated 15-character cmd name), PPID, UID, State, current and peak RSS, peak virtual size, kernel `oom_score`, operator-set `oom_score_adj`, cgroup membership, and **the parent process's command line** (so a script that forked the runaway is identified). This is the section that answers "which process actually caused the incident, owned by whom, started by what".
7. **Kernel OOM events (filtered dmesg)** — lines from the kernel ring buffer matching `Out of memory`, `oom-killer`, `Killed process`, `Memory cgroup out of memory`, or `oom_reaper`. Empty in normal operation; populated when the kernel itself terminated something.
8. **/proc/meminfo, /proc/loadavg, /proc/pressure/{memory,cpu,io}** — verbatim.
9. **User-slice cgroup** — `memory.current`, `memory.max`, etc. from `/sys/fs/cgroup/user.slice/user-<uid>.slice/`.
10. **Journal tail** — last ~200 lines of the system journal.
11. **Capture errors** — anything we could not collect, named.

Reports are atomically written (temp + rename), so a partial file is never observable.

Why §6 forensic detail matters

atop's PRM line gives only a truncated command name (~15 characters, often only the base-name) and no parent / cgroup / user / argv context. For root-cause work — "which one of the five `claude` instances on this host actually went runaway, and what was it asked to do?" — that is not enough. The forensic-detail section reads `/proc/<pid>/cmdline`, `/proc/<pid>/status`, `/proc/<pid>/cgroup`, `/proc/<pid>/oom_score{,_adj}`, and the parent's `cmdline`, then renders one subsection per top-mem PID with every field labelled. An operator opening the report two days after an incident gets the full forensic picture from this section alone.

Best-effort: a process that exits between top-N selection and detail capture leaves a benign placeholder ("*Process exited between top-N selection and detail capture*") rather than aborting the report. The number of PIDs enriched is bounded by `EnrichTopN` (default 10) so the wall-clock cost of enrichment is small even on hosts with thousands of processes.

Operational tasks

```
# Take one diagnostic report right now (also useful as a smoke test).
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -one-shot

# Tail live reports as they appear.
ls -lt /var/log/oom-watch/reports/ | head -10

# View the most recent critical report.
ls -t /var/log/oom-watch/reports/*-critical.md 2>/dev/null | head -1 | xargs less

# Validate config before reload.
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run

# Check effective config (defaults + file overlay).
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -print-config

# Service control.
sudo systemctl status oom-watch
sudo systemctl restart oom-watch
journalctl -fu oom-watch
```

Real-pressure validation (Challenge harness)

The repository ships a self-contained memory hog at `oom-watch/cmd/oommemhog/` and an end-to-end Challenge `challenges/challenge-real-pressure.sh` that exercises the entire detection pipeline against actual host pressure:

```
make oommemhog-build        # build the in-tree allocator
bash challenges/challenge-real-pressure.sh
```

`oommemhog` allocates a bounded amount of memory (capped at 16 GiB / 5 min hold) and **touches every 4 KiB page** so MemAvailable actually drops — without that, Linux's zero-page CoW would defeat the test. The Challenge:

1. Reads current host MemTotal / MemAvailable.
2. Sets thresholds dynamically (warn = current+5%, critical = current+10%) so the test triggers regardless of how much memory is currently in use.
3. Computes target = min(20% of available, 6 GiB).
4. Starts `oomwatch` in continuous mode with a unique sandbox config.
5. Runs `oommemhog` with a unique label.
6. Asserts a WARN-or-CRITICAL report was written, that it lists `oommemhog` in top-mem, and that the `/proc/meminfo` capture inside the report shows MemAvailable dropped vs. the pre-test reading.

This is the strongest anti-bluff guarantee in the repo: a parser regression that returned `MemAvailable=0` would pass fixture-tests but fail this Challenge because the report's MemAvailable would not show the expected drop.

`systemd-oomd` (when configured by `oom-hardening.sh`) is the safety backstop: any runaway gets killed at the cgroup level long before the host can stall.

Tuning

- **Lower** `interval_seconds` if you frequently miss fast-rising leaks (default 10s usually catches them).
- **Raise** `memory_used_ratio_warn` if you run sustained-high-memory workloads and don't want a report every minute. The CRITICAL level should stay near 0.95 because that's where the kernel starts swapping aggressively and PSI rises.
- **Raise** `psi_mem_full_avg10_warn` on hosts with chronic memory pressure; a value of 10% means tasks were fully stalled on memory >10% of the last 10 seconds, which is already user-visible.

atop versions and the per-thread PRM quirk

`oom-watch` was developed and field-validated against atop **2.12.1** (Sep 2025). Earlier 2.x releases work too. atop 1.x is not regularly tested; the daemon falls back gracefully but threading detail is lost.

atop 2.x emits one PRM row per kernel thread. A multi-threaded process (JVM, browser tab, modern Python with multiprocessing) appears as N rows that all share the parent's RSize because threads share an address space. atop adds a thread-group-leader flag (`y` / `n`) at field index 13 of the PRM tail; oom-watch decodes it into `PRM.IsLeader` and the report's "Top processes by resident memory" table filters by it. Without this filter a 100-thread JVM would drown out every other process in the top-N list.

The parser preserves all PRM rows (leader and non-leader) in `Sample.PRM` so future per-thread reports are possible; only the *display* layer filters. atop 1.x rows lacking the flag default `IsLeader=true` for lossless back-compat.

Failure modes and recovery

Symptom	Cause	Fix
daemon exits 1 with `atop binary not found`	atop not installed	install atop on the host
systemd unit cycles `activating → failed`, `systemctl status` shows `code=exited, status=2`	invalid `/etc/oom-watch/config.json` (unknown field, bad threshold ordering, etc.)	`oomwatch -config /etc/oom-watch/config.json -dry-run` prints the precise error. Fix the file or `rm` it and re-run `make oomwatch-deploy` to copy the current example.
daemon starts, no reports ever	thresholds set too high; or atop returns degenerate samples (zero processes)	run `oomwatch -one-shot` to verify the pipeline; check `journalctl -u oom-watch`
reports missing /proc/pressure section	host kernel < 5.15	upgrade kernel; reports remain useful without PSI
`Capture errors` lists `cgroup dir empty`	running the daemon as a non-root user, or cgroup v1 host	run as root via the systemd unit; for cgroup v1, the section is just empty
reports pile up indefinitely	no built-in rotation	use `find /var/log/oom-watch/reports -name '*.md' -mtime +30 -delete` in cron, or add a logrotate snippet

Integration with the rest of OOM-Protect

`oom-hardening.sh` sets the cgroup ceiling on `user.slice`. When the user slice approaches that ceiling, oom-watch sees `memory_used_ratio` climb (because atop's MEM uses kernel-wide `MemAvailable`, not the slice limit) AND the cgroup snapshot in the report shows `memory.current` close to `memory.max`. The two together let an investigator confirm the slice is the bound, not the box.

`oom-runner.sh` wraps individual workloads. When a wrapped process leaks, atop's PRM ranks it at the top of the per-process memory list in the report. The PID + cmd give you everything you need to `oom-runner status <unit>` or `oom-runner kill <unit>`.

Resources used

- atop official repo — parseable output format.
- atop interactive cheatsheet (PDF) — useful when running atop interactively to triage a report.
- atoptool.nl.

- bytedance/netatop-bpf — eBPF-based per-process network stats; future integration target for oom-watch's PRN label.
- pizhenwei/atophttpd — atop served over HTTP; useful pattern for exposing oom-watch reports remotely.
- atopsar-plot — retrospective plotting from atop logs; complements oom-watch's incident-time reports.

Definition of done (per Constitution)

A change to oom-watch is not complete until:

1. `make oomwatch-test` is green (`go vet` + `go test -count=1 ./...`).
2. `make challenges` is green (full E2E suite).
3. The mutation audit has been performed at least once on the changed code (temporarily break a representative production line; confirm the related test goes red; restore).
4. If the change adds a feature, a new Challenge in `challenges/` covers it.
5. Manuals (`manuals/oom-watch-manual.md`, this file) are updated; `make docs` re-rendered.